

From `mdspan` to Linear Algebra: A Modern Blueprint for a Standard C++ Tensor Library

Foundational Architecture: Leveraging `std::mdspan` as the Standard View Type

The cornerstone of any proposal for a standardized C++ tensor library must be the adoption of `std::mdspan`, a feature formally added to the C++23 standard [32](#) [68](#). Its introduction marks a pivotal moment for numerical computing in C++, providing a non-owning, zero-overhead view into multidimensional data [22](#) [32](#). The design of this new library cannot be an alternative to `std::mdspan`; it must be built upon it, using it as the canonical interface for all operations involving tensor views. This approach directly addresses the user's core problem of data exchange between disparate third-party libraries by establishing a common, standardized foundation for accessing and manipulating multidimensional arrays [35](#). By operating on `std::mdspan` objects, the proposed algebraic functions ensure they are generic over data layout and access policies, promoting interoperability and eliminating redundant data copies [27](#).

The architectural significance of `std::mdspan` lies in its policy-based design, which cleanly separates four distinct concerns: Data Storage (the underlying contiguous memory buffer), Extents (the shape or size of each dimension), Layout Mapping (how multi-dimensional indices translate to a single linear offset in memory), and Accessor Policy (how elements are retrieved from memory) [32](#). This separation allows for independent optimization and customization. For instance, the same logical data can be viewed with different extents or through different layout mappings without altering the underlying data storage. This modularity is essential for creating a flexible yet performant library. The design of `std::mdspan` was heavily influenced by the Kokkos high-performance computing project, ensuring it is architected for performance portability across diverse hardware architectures, including various CPUs and GPUs [32](#) [35](#). This heritage provides a strong guarantee that the foundational component is suitable for demanding computational science applications.

The library's algebraic operations—element-wise computations, reductions, and broadcasting—should be implemented as free functions that take `std::mdspan` objects as their primary arguments. This aligns with modern C++ practices of creating composable algorithms rather than monolithic class hierarchies. For example, a hypothetical `add` function would be declared to accept two `std::mdspan` arguments and return a result that represents the sum. The actual computation would be performed by a backend engine, but the interface remains clean and generic. This design choice ensures that the library's functionality is not tied to a specific owning container; it can operate on views of any underlying data structure, provided it can be expressed as a contiguous memory buffer with a known layout. The availability of reference implementations and experimental support in compilers like GCC and NVIDIA HPC Compiler indicates that `std::mdspan` is not merely a theoretical construct but a practical tool that is already being adopted by the scientific computing community ^{34 61 62}. Adopting this standard component immediately grounds the proposal in reality and leverages the extensive work of the ISO C++ standards committee.

Core Design Principles: Performance, Safety, and Interoperability

The success of a proposed tensor library hinges on its adherence to three core design principles: achieving performance comparable to state-of-the-art third-party libraries, ensuring robust type safety to enhance developer productivity, and guaranteeing seamless interoperability within the C++ ecosystem. These principles are not mutually exclusive but form a triad of requirements that must be addressed cohesively. The design must prioritize zero-overhead abstractions wherever possible, leveraging compile-time computation and metaprogramming to eliminate runtime costs associated with complex operations. At the same time, it must provide clear, actionable feedback through compiler errors, transforming the potential complexity of template-heavy code into an accessible and maintainable experience. Finally, the architecture must be explicitly designed to break down the barriers that currently exist between different numerical libraries, making data exchange efficient and safe.

Performance is the most critical factor for a library intended to replace established players like Eigen and Armadillo ^{11 16}. The primary mechanism for achieving high performance in tensor algebra is lazy evaluation, often implemented through expression templates ^{2 14}. This technique constructs a syntax tree representing an entire

computation chain without executing it immediately, thereby avoiding the creation of temporary objects that would otherwise require memory allocations and data movement ³⁹. Libraries like Eigen have demonstrated that this global optimization of an expression allows for superior instruction-level parallelism and cache utilization compared to naive, step-by-step execution ^{13 14}. However, expression templates are widely regarded as a "leaky abstraction," introducing significant complexity that can lead to slow compile times and deeply cryptic compiler error messages ^{1 4}. While C++20 brings improvements like Concepts that help tame some of this complexity, the inherent intricacy remains a major maintenance burden ^{8 55}. Therefore, the proposal must carefully evaluate whether traditional expression templates are the optimal path forward or if modern C++20 features like Ranges offer a more composable and maintainable alternative for building lazy evaluation engines ⁵.

Type safety and developer experience are intrinsically linked to performance. A library that is fast but difficult to use correctly is of limited value. The introduction of C++20 Concepts is a transformative development in this regard ^{10 72}. Concepts allow developers to place explicit constraints on template parameters, such as requiring that a type supports a particular operation or has a specific property ⁷¹. When these constraints are violated, the compiler can generate clear, human-readable error messages instead of the opaque cascade of substitution failures that plagued earlier template designs ^{67 76}. For the tensor library, this means defining concepts like `TensorView<T>` or `ScalarType<T>` and applying them to all algebraic functions. This ensures that users cannot accidentally pass incompatible types to an operation, and when they do, the error message will precisely state which concept was not satisfied. This dramatically lowers the barrier to entry for complex numerical programming and makes the library significantly more robust and maintainable ⁴².

Interoperability is the principle that directly addresses the fragmentation of the current C++ numerical landscape. The proliferation of libraries like Eigen, Armadillo, and xtensor necessitates frequent, often expensive, data conversions between their internal representations ¹⁵. The proposed library must make interoperability its default mode of operation. The use of `std::mdspan` as the fundamental view type is the primary enabler of this goal ³⁵. Since `std::mdspan` provides a standardized way to interpret a flat memory buffer as a multidimensional array, any library that adopts this convention can share data without copying ³². The library's design must therefore include implicit conversion constructors from the owning container type to `std::mdspan` and vice versa. Furthermore, the library's support for customizable layout mapping policies is crucial for interoperability ³⁶. For example, a library expecting column-major data can create an

`std::mdspan` with `std::layout_left` to correctly access the data from another library without requiring a transposition or copy operation ³⁶. This zero-copy philosophy extends beyond just views; the owning container should be designed to easily integrate with standard memory management facilities, allowing its data to be passed to other components of the C++ ecosystem with minimal friction.

Memory Management and Layout Policies

A comprehensive tensor library must provide a robust model for both memory management and data layout, as these are critical determinants of performance, especially concerning cache locality and hardware acceleration. The proposed library should introduce an owning container type, logically named `std::mdarray`, which serves as the counterpart to the non-owning `std::mdspan` ^{32 89}. This container would be responsible for managing its own memory buffer and lifetime, offering a complete solution from data definition to manipulation. The design of `std::mdarray` should follow the duality of dimensions seen in existing libraries like `xtensor`, supporting both compile-time and runtime dimensionality ³⁷. An instantiation templated with `std::extents` would represent a tensor with static dimensions, enabling maximum compile-time optimizations, including stack allocation of the tensor's metadata and potentially even the data itself ³⁷. Conversely, an instantiation with `std::dynamic_extents` would represent a tensor with dynamic dimensions, allocating its shape information and data buffer on the heap, similar to `xt::xarray` ³⁷. This flexibility caters to a wide range of use cases, from small, fixed-size tensors in performance-critical kernels to large, variable-sized datasets in machine learning applications.

The physical organization of data in memory, dictated by the memory layout, profoundly impacts computational efficiency. Access patterns that align with the chosen layout can lead to superior cache performance and faster execution ³⁶. The C++23 standardization of `std::mdspan` provides a standardized framework for handling layouts via layout mapping policies ³². The library should fully embrace this system, providing direct support for the standard layout policies: `std::layout_right` for row-major ordering (C-style) and `std::layout_left` for column-major ordering (Fortran-style) ^{24 36}. These two layouts are the de facto standards in the numerical computing world and are used by virtually all major libraries, including Eigen, Armadillo, and BLAS

implementations ^{18 30}. By supporting these out of the box, the library ensures compatibility and predictable performance for common use cases.

Beyond the standard layouts, the library must support custom layout policies to enable high-performance computing on specialized hardware. The policy-based design of `std::mdspan` allows users to define their own layout mapping classes, which specify how multi-dimensional indices map to a linear memory offset ³⁶. This extensibility is vital for optimizing for architectures like GPUs, which may benefit from specific tiling or strided memory access patterns ³². The proposed library should not only facilitate the creation of custom layouts but also provide a set of optimized, commonly used strided layouts as part of its standard offering. For instance, a library could expose a `strided_layout` policy that allows users to specify arbitrary strides for each dimension, mirroring the capability found in `xtensor` ³⁷. This would empower developers to fine-tune memory access for their specific hardware and algorithms without needing to delve into the complexities of writing their own layout mapping policies from scratch. The ability to specify layouts at runtime, as offered by `xtensor`'s `dynamic` layout option, should also be considered to provide maximum flexibility ³⁷.

Regarding memory allocation, the owning container `std::mdarray` must have a sensible default allocator strategy. It should use `std::allocator` by default but be designed to work seamlessly with the C++ memory resource concept hierarchy introduced in C++17. This would allow users who require specialized memory management, such as for aligned allocations needed by certain SIMD instructions or for managing memory pools in real-time systems, to plug in their own memory resources. The issue of over-aligned allocators, which is sometimes necessary when integrating with libraries like Eigen, highlights the importance of designing a memory model that is both flexible and robust ³¹. Looking toward the future, the evolution of `constexpr` in C++20 and beyond opens possibilities for more sophisticated compile-time memory management ^{45 47}. While current `constexpr` allocation is largely transient, future standards may permit more extensive compile-time memory planning, potentially allowing for the allocation of `mdarray` buffers during constant evaluation, further reducing runtime overhead ⁴³.

Algebraic Operations and Expression Semantics

The core utility of a tensor library is its collection of algebraic operations, which must be expressive, efficient, and correct. The design must cover three key areas: element-wise operations with broadcasting, reduction operations, and the underlying mechanism for expression evaluation, including the critical issue of aliasing. The proposed library should adopt a functional, free-function approach for its operators (`add`, `multiply`, etc.), making them analogous to functions in `<algorithm>`. This design promotes composability and avoids the pitfalls of operator overloading while remaining intuitive for users familiar with mathematical notation. All these operations would operate on `std::mdspan` views, ensuring they are agnostic to the underlying ownership model.

Element-wise operations and broadcasting are fundamental to modern numerical computing, popularized by libraries like NumPy and implemented in C++ by `xtensor` ⁴⁰ ⁶³. The library must provide comprehensive support for binary operations (e.g., `+`, `-`, `*`, `/`) and unary operations (e.g., `sin`, `exp`). The implementation of broadcasting—the rule that allows operations between tensors of different shapes—should follow the standard NumPy semantics. This involves inferring the output shape by comparing the dimensions of the input tensors from right to left and determining if they are compatible (i.e., one of them is of size 1 or both are of the same size). The `std::mdspan`'s flexible indexing mechanism, which uses a strided formula to calculate offsets, is perfectly suited for implementing this logic efficiently ³⁷. The layout mapping policy determines the strides, and the broadcasting algorithm can compute the resulting strides for the output tensor based on the input strides and shapes, all without moving data.

Reduction operations, such as `sum`, `mean`, `max`, and `min`, are another essential category. These operations aggregate values along specified axes of a tensor. The proposed API should mirror that of existing libraries and languages like NumPy, providing a method that takes the input `mdspan` and a set of axes along which to reduce. The result will be a new tensor with the reduced dimensions. For example, reducing a 4x3 tensor along axis 0 produces a 3-element vector. The implementation should again leverage the underlying contiguous memory and stride information to iterate through the correct elements efficiently. Eigen provides a rich set of reduction functionalities, and the new library should aim for a similarly comprehensive feature set ²⁸ ⁷⁴.

The most challenging aspect of implementing these operations is managing expression evaluation and aliasing. Aliasing occurs when the destination tensor is involved in the computation on the right-hand side of an assignment (e.g., `A = A + B`). Performing this operation in-place without a temporary buffer can lead to catastrophic data corruption,

as the source values are overwritten before they are all read. Both Eigen and xtensor have developed sophisticated strategies to handle this. Eigen treats the assignment operator `operator=` as a "barrier" that forces the evaluation of the entire right-hand side expression into a temporary before writing the result ^{17 39}. Xtensor employs a 'temporary variable rule' by default, automatically creating a temporary for any expression where aliasing might occur ³⁷. The proposed library must incorporate a similarly robust aliasing detection mechanism. This could be achieved at compile time using concepts to analyze the relationships between the input and output tensors. If aliasing is detected, the library should either implicitly create a temporary or provide a clear compile-time error, forcing the user to be explicit about their intentions (e.g., using a special `inplace` keyword if they are certain it is safe).

For lazy evaluation, the library faces a strategic choice. Traditional expression templates, while powerful, are a "leaky abstraction" and can be complex to debug ¹. The rise of C++20 features like Ranges and Coroutines offers compelling alternatives for building lazy evaluation engines with less boilerplate and greater composability ⁵. A hybrid approach might be optimal: using simple expression templates for basic arithmetic chains due to their maturity and performance, while reserving a Range-based or coroutine-based engine for more complex, chained operations. Regardless of the implementation, the goal remains the same: to build a syntax tree at compile time that captures the entire computation and execute it in a single, optimized loop at runtime, thereby eliminating intermediate temporaries and maximizing performance ².

Proposal Roadmap and Strategic Considerations

A successful proposal for inclusion in the C++ standard library must present a well-defined scope for an initial release, acknowledging that a truly "full-featured" library is an evolutionary goal. The first version of this tensor algebra proposal should focus on establishing a solid, foundational layer of functionality centered around core tensor algebra operations and the standardized memory view provided by `std::mdspan`. This initial phase should deliver a self-contained module that is performant, type-safe, and interoperable, proving its value before expanding into more complex domains. The strategic roadmap should clearly delineate what is included in the first version versus what is planned for future enhancements, demonstrating foresight and a pragmatic approach to standardization.

The initial proposal should concentrate on the core algebraic operations identified previously: element-wise binary and unary operations, broadcasting, and reduction. The primary interfaces will be free functions operating on `std::mdspan` views, ensuring maximum generality. The owning container, `std::mdarray`, should also be included, as it completes the picture of a self-contained library by providing data ownership alongside operations ³². The memory management for `std::mdarray` should be based on the standard `std::allocator`, with the design accommodating the use of custom memory resources via the C++17 memory resource API. This initial scope is substantial and sufficient to solve the immediate problems of interoperability and lack of a standard algebraic toolkit.

Looking ahead, the roadmap for the library should acknowledge several key areas for future development. The most obvious extension is the inclusion of more advanced linear algebra operations, such as matrix decompositions (LU, QR, SVD) and solvers for linear systems. The existence of a standardized tensor view and container creates a natural foundation for these higher-level algorithms. The design of the initial version should anticipate this by defining concepts related to tensor properties (e.g., `Symmetric<Tensor>`, `Diagonal<Tensor>`) that could be used to dispatch to highly optimized algorithms ²⁸. Another critical direction is hardware acceleration. The library's policy-based design, particularly its support for custom layout mapping policies, is a prerequisite for offloading computations to accelerators like GPUs ³². Future proposals could introduce execution policies or contexts that allow users to specify where an operation should be executed, with the library's backend routing the computation to an appropriate backend like one based on SYCL, CUDA, or oneAPI.

Finally, the proposal should consider the long-term evolution of the C++ language itself. Features under active consideration for future standards, such as reflection for C++26, could unlock entirely new paradigms for tensor manipulation ⁵⁹. Reflection could enable libraries to introspect tensor structures and automatically generate optimized code or even symbolic expressions. While these features are too speculative for the initial proposal, acknowledging them demonstrates that the library's design is forward-looking and adaptable. The ultimate vision, as suggested by the trajectory of standardization efforts, is a cohesive ecosystem evolving from `std::mdspan` (views) to `std::mdarray` (containers) and finally to a comprehensive linear algebra facility in a future standard like C++26, building upon the foundation laid by `mdspan` ³². By submitting a focused, well-designed first version, this proposal can initiate that important evolutionary path.

Reference

1. When should I use C++ expression templates in computational science ... <https://scicomp.stackexchange.com/questions/1634/when-should-i-use-c-expression-templates-in-computational-science-and-when-sh>
2. Expression Templates: Zero-Cost Math Abstractions in Modern C++ <https://medium.com/towardsdev/expression-templates-in-modern-cpp-4fd2c5a7f97e>
3. C++ Expression Templates for Specifying Compile-time DSP ... <https://www.youtube.com/watch?v=2F-21UUn1L4>
4. Drawbacks to templates and the STL in C++ [closed] - Stack Overflow <https://stackoverflow.com/questions/266523/drawbacks-to-templates-and-the-stl-in-c>
5. We don't need no stinking expression templates - Andy G's Blog <https://gieseanw.wordpress.com/2019/10/20/we-dont-need-no-stinking-expression-templates/>
6. Various Template Improvements with C++20 - Modernes C++ <https://www.modernescpp.com/index.php/template-improvements-with-c-20/>
7. How often is template meta programming used in real life? https://www.reddit.com/r/cpp_questions/comments/1em3gtq/how_often_is_template_meta_programming_used_in/
8. C++17 Migration of Expression Templates Library (ETL) <https://baptiste-wicht.com/posts/2018/02/c%2B%2B17-migration-of-expression-templates-library-etl.html>
9. Expression Templates in C++ - Jeremy Rifkin <https://rifkin.dev/blog/expression-templates>
10. The Complete Guide to C++20 Concepts - Dev Genius <https://blog.devgenius.io/the-complete-guide-to-c-20-concepts-revolutionizing-template-programming-419a5091f08c>
11. Eigen vs Blaze vs Fastor vs Armadillo vs XTensor - Medium <https://romanpoya.medium.com/a-look-at-the-performance-of-expression-templates-in-c-eigen-vs-blaze-vs-fastor-vs-armadillo-vs-2474ed38d982>
12. Poor performance of xtensor compared to Eigen, Blaze and Fastor <https://github.com/xtensor-stack/xtensor/issues/2269>
13. Guide to Linear Algebra With the Eigen C++ Library - Daniel Hanson <https://www.youtube.com/watch?v=99G-APJkMc0>

14. How can the C++ Eigen library perform better than specialized ... <https://stackoverflow.com/questions/10366054/how-can-the-c-eigen-library-perform-better-than-specialized-vendor-libraries>
15. Which matrix library should I use? : r/cpp - Reddit https://www.reddit.com/r/cpp/comments/ypvio0/which_matrix_library_should_i_use/
16. Armadillo and Eigen: A Tale of Two Linear Algebra Libraries - arXiv <https://arxiv.org/html/2409.00568v1>
17. Lazy Evaluation and Aliasing - Eigen <https://libeigen.gitlab.io/eigen/docs-nightly/TopicLazyEvaluation.html>
18. Eigen VS Armadillo • KDE Community Forums <https://forum.kde.org/viewtopic.php%3Ff=74&t=109532.html>
19. Eigen: A C++ template library for linear algebra | Hacker News <https://news.ycombinator.com/item?id=30906376>
20. Expressions and lazy evaluation — xtensor documentation <https://xtensor.readthedocs.io/en/latest/expression.html>
21. spanny 2: Rise of C++ std::mdspan - Griswald Brooks ... <https://www.youtube.com/watch?v=2VlK0vFZc7k>
22. C++23 std::mdspan: Multidimensional Views, Memory ... <https://medium.com/towardsdev/c-23-std-mdspan-multidimensional-views-31e98923cea5>
23. MDSPAN <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0009r17.html>
24. C++23: mdspan : r/cpp https://www.reddit.com/r/cpp/comments/15rr7a5/c23_mdspan/
25. A Look at std::mdspan - Asher's Blog <https://www.ashermancinelli.com/csblog/2021-12-23-std-mdspan-Response.html>
26. c++ - What is an mdspan, and what is it used for? <https://stackoverflow.com/questions/75778573/what-is-an-mdspan-and-what-is-it-used-for>
27. std::mdspan <https://en.cppreference.com/cpp/container/mdspan>
28. Tensor support | Eigen: A C++ template library for linear algebra https://libeigen.gitlab.io/pages/tensor_support/
29. C++23 mdspan - Alex on software development <https://alexsyniakov.com/2025/04/26/c23-mdspan/>
30. Eigen VS Armadillo • KDE Community Forums <https://forum.kde.org/viewtopic.php%3Ff>
31. Using STL Containers with Eigen https://libeigen.gitlab.io/eigen/docs-5.0/group__TopicStlContainers.html

32. The Role and Future of `mdspan` for Performance-Portable ... - Medium <https://medium.com/@dikhyantkrishnadalai/the-role-and-future-of-mdspan-for-performance-portable-multidimensional-arrays-in-c-1463c0335d66>
33. Best practice for optimum performance: multidimensional arrays - Reddit https://www.reddit.com/r/cpp/comments/1cner9k/best_practice_for_optimum_performance/
34. RFC: Adoption of `std::mdspan` like structures in C++ code #20334 <https://github.com/scipy/scipy/issues/20334>
35. Modernizing C++ Interfaces with Concepts, Constraints and `std` <https://ideas-productivity.org/events/hpcbp-096-cpp>
36. Layout Mapping in `mdspan` | Multidimensional Arrays and `std` <https://www.studyplan.dev/pro-cpp/mdspan/q/mdspan-layout-mapping>
37. Arrays and tensors — xtensor documentation - Read the Docs <https://xtensor.readthedocs.io/en/latest/container.html>
38. Introduction — xtensor documentation <https://xtensor.readthedocs.io/>
39. Eigen: Coding style's effect on performance - c++ <https://stackoverflow.com/questions/37658651/eigen-coding-styles-effect-on-performance>
40. Xtensor : the lazy tensor algebra library, by Johan Mabilie <https://www.youtube.com/watch?v=HiBhYTcmS0w>
41. Armadillo and Eigen: A Tale of Two Linear Algebra Libraries https://www.researchgate.net/publication/383701713_Armadillo_and_Eigen_A_Tale_of_Two_Linear_Algebra_Libraries
42. Template concepts in C++20 : r/cpp https://www.reddit.com/r/cpp/comments/1if5jfj/template_concepts_in_c20/
43. What's so hard about `constexpr` allocation? <https://brevzin.github.io/c++/2024/07/24/constexpr-alloc/>
44. 20 Smaller yet Handy C++20 Features <https://www.cppstories.com/2022/20-smaller-cpp20-features/>
45. `constexpr` and `constexpr` Functions in C++20 – MC++ BLOG <https://www.modernescpp.com/index.php/constexpr-and-constexpr-functions-in-c-20/>
46. c++ - How will C++20 `constexpr` containers work? <https://stackoverflow.com/questions/57163315/how-will-c20-constexpr-containers-work>
47. Design and evolution of `constexpr` in C++ <https://pvs-studio.com/en/blog/posts/cpp/0909/>
48. All C++20 core language features with examples <https://oleksandrkv1.github.io/2021/04/02/cpp-20-overview.html>
49. C++23 <https://cppreference.com/cpp/23>

50. C++23: The Next C++ Standard – MC++ BLOG <https://www.modernesccpp.com/index.php/c23-the-next-c-standard/>
51. A cheatsheet of modern C++ language and library features. ... <https://github.com/anthonycalandra/modern-cpp-features>
52. C++ conformance improvements in Visual Studio 2019 <https://learn.microsoft.com/en-us/cpp/overview/cpp-conformance-improvements-2019?view=msvc-170>
53. C++20 STL Features: One Year of Development on GitHub <https://www.youtube.com/watch?v=8kjRx8vo6y4>
54. C++23 Language Features and Reference Cards https://www.cppstories.com/2024/cpp23_lang/
55. All C++20 core language features with examples <https://news.ycombinator.com/item?id=26723851>
56. Are you a C++ Dev? Here's a plot twist ... <https://www.facebook.com/visualstudio/posts/are-you-a-c-devheres-a-plot-twist-for-you-adding-cwinrt-woke-up-a-fossilized-ope/1360818509422633/>
57. Microsoft C/C++ language conformance <https://learn.microsoft.com/en-us/cpp/overview/visual-cpp-language-conformance?view=msvc-170>
58. C++23: The Next C++ Standard <https://www.linkedin.com/pulse/c23-next-c-standard-rainer-grimm>
59. Reflection for C++26 <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2996r11.html>
60. NumKong: 2'000 Mixed Precision Kernels For All | Ash's Blog <https://ashvardanian.com/posts/numkong/>
61. NVIDIA HPC Compilers User's Guide <https://docs.nvidia.com/hpc-sdk/compilers/hpc-compilers-user-guide/index.html>
62. GCC: NEWS <https://fossies.org/linux/gcc/NEWS>
63. xtensor https://xtensor.readthedocs.io/_/downloads/en/stable/pdf/
64. Reviewing Deprecated Facilities of C++17 for C++20 <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0619r4.html>
65. C++20 <https://cppreference.com/cpp/20>
66. Modern C++ Part III: New Language Features in C++17 <https://medium.com/@afinlay/new-language-features-language-changes-in-c-17-7e953ff64c65>
67. Dangerous elements of modern C++ : r/cpp https://www.reddit.com/r/cpp/comments/fa9g4t/dangerous_elements_of_modern_c/
68. C++23 - ISciNumPy.dev <https://iscinumpy.dev/post/cpp-23/>

69. C++20: Modern Features Explained for Beginners <https://www.youtube.com/watch?v=ZQdvLQPCoao>
70. Templates Made Easy With C++20 - Roth Michaels <https://www.youtube.com/watch?v=z8e5cyZJtUw>
71. C++20 concepts require operator overloading combine ... <https://stackoverflow.com/questions/66523693/c20-concepts-require-operator-overloading-combine-with-user-define-template-op>
72. Simplify C++ templates with concepts - WWDC22 - Videos <https://developer.apple.com/videos/play/wwdc2022/110367/>
73. Blaze: A high-performance C++ math library <https://news.ycombinator.com/item?id=34407106>
74. Reductions, visitors and broadcasting https://libeigen.gitlab.io/eigen/docs-5.0/group__TutorialReductionsVisitorsBroadcasting.html
75. OpenCV vs. Armadillo vs. Eigen vs. more! Round 3 - Nghia Ho <https://nghiaho.com/?p=1726>
76. C++20 Reference Card <https://www.cppstories.com/2020/01/cpp20refcard.html/>
77. Reviewing Deprecated Facilities of C++20 for C++23 <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2139r0.html>
78. Modern C++: The C++ STL Library (Lecture 4, I. Vizzo, 2020) <https://www.youtube.com/watch?v=T2ZwqdwHRxg>
79. ISO/IEC JTC1/SC22/WG21 - Papers 2024 <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/>
80. C++23 Feature Freeze: Summer ISO WG21 Meeting Results <https://medium.com/yandex/c-23-feature-freeze-summer-iso-wg21-meeting-results-bc79d03668b3>
81. matrix multiplication - Armadillo vs Eigen3 Timing Difference <https://stackoverflow.com/questions/34622006/armadillo-vs-eigen3-timing-difference>
82. I wonder how Eigen compares to xtensor, which was ... <https://news.ycombinator.com/item?id=30907821>
83. AGU24 <https://agu.confex.com/agu/agu24/meetingapp.cgi/Paper/1656898>
84. Publications <https://wantongli.github.io/Publications/>
85. Untitled <https://ui.adsabs.harvard.edu/abs/2026EPSR..25312513N/abstract>
86. Research - Boxun Xu <https://boxunxu.top/publications/>
87. SD-1: 2021 PL22.16/WG21 Document List <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/sd-1.htm>
88. Standardization <https://isocpp.org/blog/category/standardization/P10>

89. A C++ Approach to Physical Units - Mateusz Pusz - CppCon 2019 <https://www.youtube.com/watch?v=0YW6yxkdhlU>